HLD Level 1.10

Observability & Performance — C++ / Systems Interview Notes

> **Core question: the system is slow or unhealthy. How do you identify where the problem is instead of guessing? Use logs, metrics, traces, profiling and resource/saturation measurements.**

1. Observability

Observability is the ability to understand internal system behavior from externally available signals. The classic three pillars are logs, metrics and traces.

```
System -> Logs + Metrics + Traces
```

2. Logs — what happened?

Logs capture discrete events: startup, connection failures, retries, exceptions, state transitions and important business/system events. Useful context includes timestamp, severity, component, request/device ID, event, error code and relevant latency.

```
10:01:23 device=42 component=heartbeat event=timeout attempt=3
```

3. Structured logging

Structured fields are easier to search and aggregate than free-form sentences. Avoid logging enormous payloads or secrets, and control high-frequency logging because it can become an I/O/performance problem.

4. Log levels

Typical levels include DEBUG, INFO, WARN and ERROR. Production logging should make abnormal behavior discoverable without drowning operators in noise.

5. Metrics — how is the system behaving?

Metrics are numerical measurements over time. Common service metrics include throughput, error rate, latency, CPU, memory, queue depth, worker utilization, active connections and retry rate.

```
FPS=30; inference=40ms; queue_depth=12; CPU=76%; errors=0.2%
```

6. Traffic, errors, latency, saturation

A useful interview framework is to inspect traffic/load, error rate, latency and resource saturation. For queues, also inspect depth and wait time.

7. Percentiles

Latency distributions are better represented by percentiles than averages alone.

```
P50 = 40 ms
P95 = 75 ms
P99 = 350 ms
```

P50 is the median. P95 means about 95% of observations are at or below that value. P99 exposes tail behavior experienced by the slowest ~1% of requests.

8. Why averages mislead

```
99 requests x 40ms + 1 request x 2000ms
average ~= 59.6ms
```

The average looks reasonable even though one request took two seconds. Tail percentiles expose spikes hidden by averages.

9. Tail latency

P95/P99 spikes can come from queueing, lock contention, scheduling, page faults, network retries, disk I/O, garbage/allocator behavior, downstream dependencies or overloaded pools.

10. Tracing — where did the time go?

Distributed tracing follows one request through multiple components and records spans/timing.

```
Gateway 20ms -> Order 40ms -> Payment 760ms -> DB 50ms
```

If total latency is 900 ms, tracing quickly identifies the dominant service/path rather than searching every component independently.

11. Correlation IDs

Propagate a request/trace ID through services and include it in logs so events belonging to one operation can be reconstructed.

```
ABC123 Gateway received
ABC123 ServiceB timeout
ABC123 retry
ABC123 returned 500
```

12. Queue depth

Queue depth is a critical leading indicator. A steadily rising queue means arrival rate exceeds processing capacity or downstream work has slowed.

```
depth: 0 -> 5 -> 12 -> 20 -> 35 -> 50 -> 60
```

13. Queue wait time

Depth alone is not enough: measure how long items wait before processing. A queue can be numerically small yet violate latency targets when processing is slow.

14. Stage-level instrumentation

Measure each pipeline stage instead of only end-to-end time.

```
Capture 5ms
Preprocess 8ms
Inference 30ms
Postprocess 7ms
Total ~50ms
```

This makes optimization evidence-driven and helps identify whether the bottleneck is compute, queueing, I/O or synchronization.

15. steady_clock in C++

For elapsed-duration measurements in C++, std::chrono::steady_clock is appropriate because it is monotonic and is not intended to jump with wall-clock adjustments.

```
auto start = std::chrono::steady_clock::now();
// operation
auto end = std::chrono::steady_clock::now();
```

16. Profiling

Profiling identifies where CPU time, allocations, memory, locks or I/O are consumed. Observability tells you that something is wrong; profiling can identify expensive code inside the process.

17. CPU profiling

If CPU is saturated, determine which functions or code paths consume time before optimizing. Optimize the dominant cost rather than random code.

```
Function A = 55% CPU
Function B = 12% CPU
Function C = 3% CPU
```

18. Memory debugging/profiling

For C++, monitor RSS/heap growth, allocation rate, leaks, buffer pools and queue memory. Tools such as AddressSanitizer and Valgrind help detect classes of memory errors/leaks; profiling tools answer different performance questions.

19. Bottleneck analysis

Use a repeatable loop: measure baseline, identify bottleneck, form a hypothesis, optimize, then measure again.

```
MEASURE -> IDENTIFY -> OPTIMIZE -> MEASURE AGAIN
```

Do not start with 'add servers' or 'rewrite code.' First identify the constrained resource or slow stage.

20. CPU saturation

High CPU together with increasing queue depth and latency suggests compute saturation. Potential responses include optimizing expensive code, reducing work, batching, using available hardware acceleration, or scaling horizontally.

Adding threads beyond available compute can increase context switching and lock contention instead of improving throughput.

21. Memory budgeting

```
70 frames x 3MB ~= 210MB payload alone
```

Real memory also includes additional pipeline buffers, encoded data, STL/container overhead, allocator metadata, libraries, stacks and kernel/network buffers.

22. Network metrics

For network-heavy systems, monitor active connections, connection rate, disconnect/reconnect rate, timeout rate, failed connections and bytes/sec.

23. Reconnect storms

```
normal reconnects = 10/sec
incident reconnects = 15,000/sec
```

A sudden reconnect spike can overload TLS/authentication/connection setup and is a valuable incident signal.

24. Error rate

Track error rate and error categories separately: timeouts, 5xx, connection failures, dependency errors and invalid requests. A single aggregate error number can hide the cause.

```
error_rate = errors / total_requests
```

25. Saturation

Saturation means a resource is approaching or at capacity.

```
CPU 98%
DB pool 100/100
Queue 68/70
Workers 32/32 busy
FDs near limit
```

Saturation metrics often warn of failure before the system actually becomes unavailable.

26. Alerts

Alerts should be actionable and avoid excessive noise. A sustained error rate or P99/queue-depth condition is often more useful than a brief CPU spike.

```
error_rate > 5% for 5 min
or
P99 above target + queue continuously rising
```

27. SLI, SLO, SLA

SLI: measured indicator, such as availability or P99 latency.
SLO: target for that indicator.
SLA: formal service commitment/agreement, potentially with business consequences.

```
SLI = measurement
SLO = target
SLA = formal commitment
```

28. Performance testing

Use representative load and realistic request/data distributions. Test average and peak load, sustained load, bursts and failure conditions. Warm-up effects and cache state can influence results.

29. Capacity vs latency

As utilization approaches a resource's limit, queueing and tail latency can rise sharply. Do not design only for theoretical maximum throughput; preserve headroom for bursts and failures.

30. Lock contention metrics

For multithreaded C++ services, inspect time waiting for locks, critical-section duration and whether one shared lock serializes otherwise parallel work.

31. Allocation/copying costs

High allocation rates and repeated copies of large buffers can consume CPU and memory bandwidth. Reuse buffers or move ownership when profiling justifies it.

32. Disk and I/O

If CPU is low but latency is high, inspect disk/network wait, downstream services and queueing. Low CPU does not mean the system has spare end-to-end capacity.

33. Your camera optimization story

A strong interview explanation is: instrument each stage using monotonic timestamps, compare baseline and worst-case/tail behavior, identify dominant costs and backlog, optimize, then repeat measurements. Your observed improvement from roughly 55 ms / 21 FPS toward roughly 40 ms / 30 FPS is stronger when presented as a measurement-driven process rather than simply 'rewrote it in C++.'

34. Investigating a slow API

Start with request rate and P50/P95/P99 -> error rate -> CPU/memory -> queue depth/wait -> worker/connection pools -> dependency latency -> traces -> correlated logs -> in-process profiler if needed.

35. Observability overhead

Instrumentation itself costs CPU, memory, network and storage. Avoid excessive high-cardinality metrics, unbounded labels, giant logs and tracing every request at full detail when sampling is sufficient.

36. Cardinality

A metric label such as device_id across millions of devices can create huge cardinality and storage/query cost. Use logs/traces for highly specific identifiers and keep metrics labels controlled.

37. Interview checklist

1. What are the SLIs/SLOs?
2. What throughput and latency metrics?
3. P50/P95/P99?
4. Error categories?
5. Queue depth/wait?
6. CPU/memory/FD/connection saturation?
7. Correlation/trace IDs?
8. Distributed tracing?
9. Stage timings?
10. Profiling strategy?
11. Alerts?
12. What does success look like after optimization?

38. Common mistakes

Only average latency: hides tail spikes.
Log everything: expensive/noisy.
High-cardinality metric labels: costly.
Optimize before measuring: may target irrelevant code.
CPU is low so system is healthy: could be blocked on I/O/locks.
CPU is high so add threads: may worsen saturation.
Only end-to-end timing: does not locate the stage.
No correlation ID: hard to reconstruct distributed requests.
Alert on every spike: creates alert fatigue.

39. Practice scenarios

1. Average latency is 60 ms but users report occasional 2-second requests. What metrics do you inspect?

2. CPU is 40%, but queue depth rises continuously. What possibilities would you investigate?

3. A request crosses five services and takes 1.2 seconds. Which observability signal best shows where time was spent?

4. Camera inference occasionally spikes from 40 ms to 200 ms. How would you instrument the pipeline?

5. Why can device_id be dangerous as a metrics label for millions of devices?

6. Queue is 68/70, workers are all busy and P99 is rising. What does this suggest?

7. How would you prove an optimization actually improved the system?

40. 60-second revision

Logs: what happened.
Metrics: how system behaves over time.
Traces: where request time went.
P50: median.
P95/P99: tail latency.
Queue depth/wait: backlog/saturation signals.
Profiling: find expensive code/resources.
SLI: measurement.
SLO: target.
SLA: formal commitment.
Golden rule: measure -> identify -> optimize -> measure again.

41. Level 1 complete

You have now covered all ten Level 1 foundations: requirements/capacity, networking, load balancing, caching, storage, replication/sharding, messaging, reliability, concurrency/backpressure, and observability/performance. The next stage is Level 2: complete system-design problems.